

Software Metrics – Assignment 6

1. Repository

The software repository monitored in this assignment is the FastAPI open-source project:

<https://github.com/fastapi/fastapi>

The repository was selected because it is an active open-source software project with a long commit history, multiple contributors, and continuous development activity, making it suitable for software measurement and mining software repositories (MSR) analysis.

2. Data

For this assignment, we monitored data from the FastAPI repository. The repository contains an active development history with thousands of commits and contributions from multiple developers.

The measurement system extracts software repository data directly from the Git history of the project. The collected data include:

- commit history
- modified files
- developer activity
- file change frequency
- code complexity information

The repository data were collected automatically using Python scripts and Git mining techniques. Metrics were computed using a rolling 30-day analysis window in order to support longitudinal software evolution analysis.

3. Metrics

Metric	Definition	Citation
Stability Index	Measures software stability based on how frequently files are modified over time. Frequently modified files indicate lower software stability.	Hassan (2009)
Change Frequency	Measures the number of commits occurring within a defined time window to capture repository activity and software evolution trends.	Kalliamvakou et al. (2014)
Code Ownership	Measures the concentration of contributions among developers. High ownership indicates that a small number of developers dominate modifications to the codebase.	Bird et al. (2011)

Developer Activity	Measures the number of active contributors participating in repository development during a given time window.	Vasilescu et al. (2015)
Change Coupling	Measures how frequently files are modified together within the same commits. High coupling may indicate architectural dependencies between components.	Gall et al. (2003)
Cyclomatic Complexity	Measures structural complexity of source code based on the number of independent execution paths in the program.	McCabe (1976)
Hotspot Index	Measures concentration of development activity on a subset of files. Frequently modified files are considered maintenance hotspots.	Tornhill (2014)
Code Review Activity	Measures repository review participation through pull request review interactions and discussions during development.	Bacchelli and Bird (2013)
Continuous Integration Activity	Measures the frequency of CI/CD executions and automated workflow activity in the repository development process.	Hilton et al. (2016)
Lead Time for Changes	Measures the time required for code changes to move from commit creation to integration into the repository.	Forsgren, Humble, and Kim (2018)

4. Dashboard and Metric Computation

To compute and visualize software metrics, we implemented a Python-based measurement pipeline that extracts data from the Git history of the *FastAPI* repository and stores computed values in a structured CSV file. The metrics are computed within a sliding time window of 30 days to support longitudinal analysis of software evolution.

All metrics are implemented based on definitions commonly used in Mining Software Repositories (MSR) research and are calculated programmatically using Git commit history and file-level changes.

- **Stability Index**

Stability measures how frequently files are modified within a given time window. Files that are modified more frequently indicate lower stability of the system.

$$Stability = \frac{| \text{Modified Files} |}{| \text{Total Unique Files} |}$$

- **Change Frequency**

Change frequency measures the number of commits occurring within the analysis window. It reflects the development intensity of the repository.

$$ChangeFrequency = |C|$$

where C is the set of commits in the time window.

- **Code Ownership**

Code ownership measures how contributions are distributed among developers. It is computed as the proportion of contributions made by the most active developer.

$$Ownership = \frac{\max(\text{commits per developer})}{\text{total commits}}$$

- **Developer Activity**

Developer activity measures the number of unique contributors active within the time window.

$$DevActivity = |\{\text{authors of commits}\}|$$

- **Change Coupling**

Change coupling measures how often files are modified together in the same commit, indicating logical dependencies between files.

$$Coupling = \frac{\sum_{c \in C} \binom{n_c}{2}}{|C|}$$

where n_c is the number of files changed in commit c .

- **Cyclomatic Complexity**

Cyclomatic complexity measures structural complexity of code based on control flow structure. It reflects the number of independent execution paths in the program.

$$Complexity = \sum_{f \in F} CC(f)$$

where $CC(f)$ is the complexity of file f .

- **Hotspot Index**

Hotspot index measures concentration of changes in a small subset of files. A higher value indicates that few files receive most changes.

$$Hotspot = \frac{\max(\text{file change counts})}{\sum \text{file change counts}}$$

- **Code Review Activity**

Code Review Activity measures development quality using proxy signals of review behavior. Since direct pull request review data is not available in a pure Git-based analysis, it is approximated using merge commits in the repository history. A higher number of merge commits indicates more frequent integration of reviewed changes into the main branch.

$$CRA = |\{c \in C : c \text{ is a merge commit}\}|$$

where C is the set of commits in the time window.

- **Continuous Integration Activity**

Continuous Integration Activity measures the level of automated integration activity in the repository. In this Git-based implementation, it is approximated using integration patterns such as merge frequency and commit integration density within the analysis window. Higher values indicate more frequent integration of changes into the main branch.

$$CIA = |C_{integration}|$$

where $C_{integration}$ represents commits identified as integration or merge-related commits within the time window.

- **Lead Time for Changes**

Lead Time for Changes measures the time required for a change to move from its creation to integration into the main branch. In an ideal CI/CD environment, this is computed using timestamps from commit creation and merge completion. In this Git-based approximation, it is computed only when merge information is available.

$$LeadTime = t_{merge} - t_{commit}$$

where t_{commit} is the commit timestamp and t_{merge} is the time the change is integrated into the main branch.

If merge timestamps are not available in the dataset, this metric is treated as not directly observable in the current implementation.

These last three metrics are approximated using Git-level data due to the absence of direct CI/CD and pull request API integration in the current measurement system.

4.1 Data Collection Process

The measurement system extracts data from a local clone of the *FastAPI* repository. The extraction process uses Git logs to retrieve historical development information, including:

- commit history
- file modification lists per commit
- developer identities (commit authors)
- file-level change distributions across commits

The extraction is implemented in Python using Git command-line operations executed programmatically. A sliding time window (30 days) is applied during each execution to ensure that the metrics reflect recent development activity rather than full repository history.

The collected raw data is transformed into structured metrics and appended daily to a CSV file (metrics.csv), which serves as a longitudinal dataset for tracking software evolution over time.

4.2 Dashboard Implementation

The computed metrics are visualized using a Streamlit-based interactive dashboard. The dashboard is designed to support longitudinal analysis of software evolution by presenting both aggregated and time-series views of the data.

The dashboard provides the following components:

- KPI overview of the most recent metric values, representing the current state of the repository
- time-series visualizations showing the evolution of each metric over time
- raw data table for full transparency and reproducibility of results

Each metric is plotted using data stored in metrics.csv, which is updated daily by the measurement pipeline. This enables continuous tracking of software evolution and supports comparative analysis across different time points.

The dashboard allows stakeholders to identify trends such as increasing complexity, changes in development activity, and concentration of changes in specific parts of the system.



4.3 Dashboard Snapshot After 15 Days of Data Collection

Figure 1 presents the software metrics dashboard after approximately fifteen days of continuous data collection from the FastAPI repository. The dashboard displays both the latest metric values and their evolution over time. The results indicate that the repository remained highly stable throughout the observation period, with the Stability Index consistently equal to 1.0. Change Frequency fluctuated between approximately 108 and 144 commits within the analysis window, indicating ongoing development activity and regular code modifications. Code Ownership remained relatively stable, ranging between 0.49 and 0.50, suggesting that development contributions were distributed among multiple contributors rather than being dominated by a single developer. Developer Activity also remained consistent, indicating sustained participation from active contributors. The Change Coupling metric showed moderate variation over time, suggesting that groups of files were frequently modified together, which may reflect logical dependencies between components of the system. Cyclomatic Complexity remained largely stable, with only minor increases observed during the monitoring period. This suggests that the repository evolved without substantial increases in structural complexity. The Hotspot Index exhibited a gradual decrease throughout the observation period, indicating that development activity became more evenly distributed across files rather than being concentrated in a small subset of the codebase. Such a trend may suggest improved maintainability and reduced dependency on specific files. Overall, the dashboard demonstrates how continuous measurement can provide insights into software evolution, development practices, and potential maintenance risks within an active open-source project.

References

- Bacchelli, A., and Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. Proceedings of the 35th International Conference on Software Engineering.
- Bird, C., Nagappan, N., Murphy, B., Gall, H., and Devanbu, P. (2011). Don't Touch My Code! Examining the Effects of Ownership on Software Quality. Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering.
- Forsgren, N., Humble, J., and Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.
- Gall, H., Jazayeri, M., and Krajewski, J. (2003). CVS Release History Data for Detecting Logical Couplings. Proceedings of the International Workshop on Principles of Software Evolution.
- Hassan, A. E. (2009). Predicting Faults Using the Complexity of Code Changes. Proceedings of the 31st International Conference on Software Engineering.

Hilton, M., Nelson, N., Tunnell, T., Marinov, D., and Dig, D. (2016). Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects. Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering.

Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D. M., and Damian, D. (2014). The Promises and Perils of Mining GitHub. Proceedings of the 11th Working Conference on Mining Software Repositories.

McCabe, T. J. (1976). A Complexity Measure. IEEE Transactions on Software Engineering, 2(4), 308–320.

Tornhill, A. (2014). Your Code as a Crime Scene: Use Forensic Techniques to Arrest Defects, Bottlenecks, and Bad Design in Your Programs. Pragmatic Bookshelf.

Vasilescu, B., Yu, Y., Wang, H., and Filkov, V. (2015). Quality and Productivity Outcomes Relating to Continuous Integration in GitHub. Proceedings of the 10th Joint Meeting on Foundations of Software Engineering.